



Nasz Zestaw Narzędziowy

Opis Funkcji

Cześć! Witajcie na warsztatach z analizy danych. Aby ułatwić Wam pracę i pozwolić skupić się na najciekawszych zadaniach, przygotowaliśmy zestaw gotowych funkcji w Pythonie. Pomyślcie o nich jak o super-narzędziach, które wykonują za Was nudne i powtarzalne czynności.

Poniżej znajdziecie instrukcję obsługi każdego z tych narzędzi.

Spis treści

 Sekcja 1: Przygotowanie i Czyszczenie Danych	2
wczytaj_dane(sciezka)	2
polacz_dane(dane1, dane2, klucz, jak)	2
usun_outliery(dane, kolumny, wspolczynnik_iqr)	3
usun_nan(dane, kolumna)	4
czyszczenie_procenty(df, col)	4
 Sekcja 2: Przekształcanie i Analiza Danych	5
skategoryzuj_kolumne_numeryczna(dane, nazwa_kolumny, ...)	5
filtruj_dane(dane, filtry)	5
top10_wartosci(df, col_text, col_value, granica)	6
 Sekcja 3: Wizualizacja Danych	7
wykres liniowy(...)	7
wykres punktowy(...)	7
wykres_babelkowy(...)	7
wykres_korelacji(...)	8
tabela_korelacji(...)	8
wykres_kolowy(...)	8
wykres_histogram(...)	9

Sekcja 1: Przygotowanie i Czyszczenie Danych

Każda praca z danymi zaczyna się od "sprzątania". Dane, które dostajemy, rzadko są idealne. Mogą mieć błędy, brakujące wartości lub po prostu być w niewygodnym formacie. Poniższe funkcje pomogą nam to wszystko ogarnąć.

wczytaj_dane(sciezka)

- **W pigułce:** Wczytuje dane z pliku `.csv` i od razu je trochę "sprząta".
- **Do czego służy?** To Wasz punkt startowy. Zamiast używać standardowej funkcji z biblioteki `pandas`, użyjcie tej. Dlaczego? Bo nasza funkcja dodatkowo:
 1. **Standaryzuje nazwy kolumn:** Zamienia wielkie litery na małe, usuwa zbędne spacje i zamienia dziwne znaki na podkreślenia (np. `_`). Dzięki temu nazwa `Rok Produkcji` stanie się `rok_produkcji`, co jest o wiele wygodniejsze w kodzie.
 2. **Usuwa niepotrzebne kolumny:** W naszych zbiorach danych często pojawia się kolumna `rank`, która nie jest nam potrzebna do analizy, więc funkcja automatycznie ją usuwa.

- **Jak używać?**

```
# Wczytujemy dane z pliku 'dane.csv' i przypisujemy je do
# zmiennej df
df = wczytaj_dane('dane.csv')
```

- **Parametry:**

- `sciezka` (`str`): Ścieżka do pliku, który chcecie wczytać, podana jako tekst (w cudzysłowie).

- **Co zwraca?** Ramkę danych `pandas.DataFrame` gotową do dalszej pracy.

polacz_dane(dane1, dane2, klucz, jak)

- **W pigułce:** Łączy dwie tabelki (ramki danych) w jedną, jak puzzle.
- **Do czego służy?** Wyobraźcie sobie, że w jednej tabeli macie listę krajów i ich populację, a w drugiej te same kraje i ich PKB. Aby mieć wszystko w jednym miejscu, musicie je połączyć. Ta funkcja robi to za Was, znajdując wspólne elementy (tzw. `klucz`).
- **Jak używać?**

```
# Mamy dwie ramki danych: df_populacja i df_pkb
# Obie maja kolumnę 'country', która posłuży jako klucz do
# polaczenia
```

```
df_polaczone = polacz_dane(df_populacja, df_pkb,
                           klucz='country')
```

- **Parametry:**

- `dane1` (`DataFrame`): Pierwsza ramka danych.
- `dane2` (`DataFrame`): Druga ramka danych.
- `klucz` (`str` lub `List[str]`): Nazwa kolumny (lub lista kolumn), która jest wspólna dla obu tabel i posłuży do ich połączenia.
- `jak` (`str`): Sposób łączenia. Domyślnie jest to `"inner"`, co oznacza, że w wyniku znajdą się tylko te wiersze, które miały parę w obu tabelkach.

- **Co zwraca?** Nową, połączoną ramkę danych `DataFrame`.

`usun_outliery(dane, kolumny, wspolczynnik_iqr)`

- **W pigułce:** Znajduje i usuwa "dziwne", mocno odstające wartości z danych.
- **Do czego służy?** Czasem w danych zdarzają się pomyłki. Np. w kolumnie `wiek` wszyscy mają wartości od 10 do 80 lat, ale jedna osoba ma wpisane `300`. To prawdopodobnie błąd, który zaburzy nam wyniki (np. średnią). Ta funkcja, używając metody statystycznej (IQR), identyfikuje takie "odstające" wartości i usuwa całe wiersze, w których się znajdują.
- **Jak używać?**

```
# Chcemy usunac outliery z kolumn 'pkb' i 'populacja'
df_czyste = usun_outliery(df, kolumny=['pkb', 'populacja'])

# Jesli nie podamy kolumn, funkcja sama sprawdzi wszystkie
# kolumny liczbowe
df_czyste_auto = usun_outliery(df)
```

- **Parametry:**

- `dane` (`DataFrame`): Ramka danych do sprawdzenia.
- `kolumny` (`List[str]`, opcjonalnie): Lista kolumn, które mają zostać sprawdzone. Jeśli nie podasz, funkcja sama znajdzie wszystkie kolumny z liczbami.
- `wspolczynnik_iqr` (`float`): Parametr dla zaawansowanych. Kontroluje, jak "wrażliwy" jest wykrywacz outlierów. Domyślnie `1.5`.

- **Co zwraca?** Ramkę danych `DataFrame` bez wierszy zawierających outliery.

usun_nan(dane, kolumna)

- **W pigułce:** Usuwa wiersze, w których brakuje danych w konkretnej kolumnie.
- **Do czego służy?** `NaN` (Not a Number) to specjalna wartość oznaczająca "brak danych". Jeśli np. analizujemy zarobki, a w tej kolumnie dla jakiejś osoby jest puste pole (`NaN`), to nie możemy obliczyć średniej. Ta prosta funkcja usuwa wszystkie wiersze, które mają `NaN` we wskazanej przez nas kolumnie.
- **Jak używać?**

```
# Usuamy wszystkie wiersze, gdzie w kolumnie 'zarobki'
# brakuje danych
df_bez_brakow = usun_nan(df, kolumna='zarobki')
```

- **Parametry:**
 - `dane` (`DataFrame`): Twoja ramka danych.
 - `kolumna` (`str`): Nazwa kolumny, którą chcesz "wyczyścić" z braków.
- **Co zwraca?** Ramkę danych `DataFrame` bez brakujących wartości w danej kolumnie.

czyszczenie_procenty(df, col)

- **W pigułce:** Zamienia tekst z procentami (np. `"98,5%"`) na prawdziwą liczbę (`98.5`).
- **Do czego służy?** Czasami dane procentowe są wczytywane jako tekst, zwłaszcza jeśli używają przecinka zamiast kropki. Ta funkcja zamienia przecinki na kropki i konwertuje całą kolumnę na typ liczbowy, aby można było na niej wykonywać obliczenia.
- **Jak używać?**

```
# Poprawiamy kolumnę 'procent_szczepien', która jest tekstem
df = czyszczenie_procenty(df, 'procent_szczepien')
```

- **Co zwraca?** Tę samą ramkę danych, ale ze zmienioną kolumną.

🧩 Sekcja 2: Przekształcanie i Analiza Danych

Gdy dane są już czyste, możemy zacząć je modyfikować i filtrować, aby przygotować je do wizualizacji i wyciągania wniosków.

`skategoryzuj_kolumne_numeryczna(dane, nazwa_kolumny, ...)`

- **W pigułce:** Zamienia liczby w kolumnie na kategorie (np. wiek 25 na grupę "20-29").
- **Do czego służy?** Analiza danych na dokładnych liczbach bywa trudna. Czasem łatwiej jest patrzeć na grupy. Ta funkcja pozwala zamienić kolumnę z liczbami na kolumnę z etykietami.
- **Jak używać?**

Tryb 1: Automatyczne grupy co stałą wartość

```
# Grupujemy wiek w przedziały co 10 lat (0-9, 10-19, 20-29 itd.)
df_z_grupami = skategoryzuj_kolumne_numeryczna(df,
    nazwa_kolumny='wiek', co_ile=10)
```

Tryb 2: Własne, niestandardowe grupy

```
# Tworzymy własne grupy dla punktów w skali 1-100
moje_grupy = {'0': '50', '51': '80', '81': '100'}
# Wynik: '0-50', '51-80', '81-100'
df_z_grupami = skategoryzuj_kolumne_numeryczna(df, 'wynik',
    grupy_niestandardowe=moje_grupy)
```

- **Co zwraca?** Nową ramkę danych `DataFrame` z przekształconą kolumną.

`filtruj_dane(dane, filtry)`

- **W pigułce:** Wybiera z tabeli tylko te wiersze, które spełniają Twoje warunki.
- **Do czego służy?** To jak filtr w Excelu. Chcesz zobaczyć tylko kraje z Europy? Albo tylko te, których populacja jest większa niż 50 milionów? Ta funkcja jest do tego idealna. Warunki podajesz w specjalnym formacie zwanym "słownikiem".
- **Jak używać?**

```
# Definiujemy nasze warunki filtrowania
moje_filtry = {
    'region': ['==', 'Europe'],          # Region musi być równy
    'Europe',
    'population': ['>', 50000000]        # Populacja musi być
    'większa niż 50 mln'
}
```

```
# Stosujemy filtry do naszej ramki danych
df_europa = filtruj_dane(df, filtry=moje_filtry)
```

- **Obsługiwane operatory:** `==` (równy), `!=` (nierówny), `>`, `<`, `>=`, `<=`, `in` (zawarty w liście), `not in` (niezawarty w liście).
- **Co zwraca?** Nową, mniejszą ramkę danych `DataFrame`, która zawiera tylko odfiltrowane wiersze.

```
top10_wartosci(df, col_text, col_value, granica)
```

- **W pigułce:** Wybiera `N` największych wartości, a resztę sumuje i wrzuca do kategorii "Inne".
- **Do czego służy?** Idealne do tworzenia czytelnych wykresów kołowych. Jeśli masz dane dla 100 krajów, pokazywanie wszystkich na wykresie byłoby nieczytelne. Ta funkcja wybierze np. 10 krajów z największą populacją, a populację pozostałych 90 zsumuje i pokaże jako jedną pozycję "Inne".
- **Jak używać?**

```
# Wybieramy 7 krajow o największej populacji, reszte
  grupujemy
df_top7 = top10_wartosci(df, col_text='country',
    col_value='population', granica=7)
```

- **Co zwraca?** Małą ramkę `DataFrame` gotową do narysowania wykresu.

Sekcja 3: Wizualizacja Danych

Liczby w tabelach są ważne, ale obraz mówi więcej niż tysiąc słów! Poniższe funkcje służą do tworzenia różnych rodzajów estetycznych i czytelnych wykresów.

Wspólne cechy funkcji rysujących:

- Nic nie zwracają – po prostu **wyświetlają gotowy wykres**.
- Mają wiele opcjonalnych parametrów (`rozmiar`, `paleta`, `tytuł` itd.), które pozwalają dostosować wygląd wykresu.

wykres liniowy(...)

- **Rodzaj wykresu:** Liniowy.
- **Kiedy używać?** Idealny do pokazywania **zmian czegoś w czasie**, np. wzrostu temperatury na przestrzeni lat, zmiany PKB, notowań giełdowych. Oś X to zazwyczaj czas (np. rok).
- **Przykład:**

```
wykres liniowy(df, kolumna_1='year',  
               kolumna_2='gdp_per_capita', grupa='country')
```

wykres punktowy(...)

- **Rodzaj wykresu:** Punktowy (rozrzutu, scatter plot).
- **Kiedy używać?** Aby sprawdzić, czy istnieje **zależność (korelacja) między dwiema zmiennymi liczbowymi**. Np. czy jest związek między wzrostem a wagą? Czy kraje z wyższym PKB mają wyższą oczekiwaną długość życia?
- **Przykład:**

```
wykres punktowy(df, kolumna_1='literacy',  
                 kolumna_2='gdp_per_capita', grupa='region')
```

wykres bąbelkowy(...)

- **Rodzaj wykresu:** Bąbelkowy (wariacja punktowego).
- **Kiedy używać?** Kiedy chcesz pokazać zależność między **trzema zmiennymi naraz**. Dwie zmienne to osie X i Y (jak w wykresie punktowym), a trzecia zmienna jest reprezentowana przez **rozmiar bąbelka**. Np. oś X to PKB, oś Y to długość życia, a rozmiar bąbelka to populacja kraju.

- **Przykład:**

```
wykres_babelkowy(df, kolumna_x='gdp_per_capita',  
                  kolumna_y='life_expectancy',  
                  kolumna_rozmiar='population', grupa='region')
```

wykres_korelacji(...)

- **Rodzaj wykresu:** Mapa ciepła (heatmap).
- **Kiedy używać?** Aby zobaczyć **siłę korelacji między wszystkimi parami zmiennych liczbowych** w Twoich danych. Wykres pokazuje tabelę, gdzie kolory oznaczają siłę zależności: od -1 (silna zależność odwrotna, kolor zimny) do +1 (silna zależność prosta, kolor ciepły). Wartości bliskie 0 oznaczają brak zależności.
- **Przykład:**

```
# Funkcja sama znajdzie wszystkie kolumny liczbowe w ramce df  
wykres_korelacji(df)
```

tabela_korelacji(...)

- **Rodzaj wykresu:** Uproszczona mapa ciepła.
- **Kiedy używać?** Gdy nie interesują Cię korelacje "wszystkiego ze wszystkim", a jedynie **zależność kilku wybranych zmiennych z całą resztą**. Np. chcesz sprawdzić, co najmocniej wpływa na `oczekiwana_dlugosc_zycia`.
- **Przykład:**

```
# Sprawdzamy, co koreluje z dlugoscia zycia i PKB  
tabela_korelacji(df, kolumny=['life_expectancy',  
                              'gdp_per_capita'])
```

wykres_kolowy(...)

- **Rodzaj wykresu:** Kołowy.
- **Kiedy używać?** Do pokazywania **proporcji i udziału procentowego** poszczególnych kategorii w całości. Np. jaki procent populacji świata mieszka na poszczególnych kontynentach. Najlepiej używać go z funkcją `top10_wartosci`!
- **Przykład:**

```
# Najpierw przygotowujemy dane  
df_top_kontynenty = top10_wartosci(df, col_text='continent',  
                                   col_value='population', granica=5)
```



```
# A potem rysujemy wykres
wykres_kolowy(df_top_kontynenty, category_col='continent',
              value_col='population')
```

wykres_histogram(...)

- **Rodzaj wykresu:** Histogram.
- **Kiedy używać?** Do pokazania **rozkładu jednej zmiennej liczbowej**. Pokazuje, jak często występują wartości z określonych przedziałów. Np. ilu jest uczniów w przedziałach wzrostu 160-165 cm, 165-170 cm, 170-175 cm itd.
- **Przykład:**

```
# Tworzymy histogram dla kolumny 'wzrost' z 15 słupkami
  (przedziałami)
wykres_histogram(df, col='wzrost', bins=15)
```